

Compte rendu TP 02.1 = 2.3 : Phase 1

Sommaire

Introduction.....	1
Préparation.....	2
I. Les deux types de reconnaissance.....	2
a) La reconnaissance passive.....	2
b) La reconnaissance active.....	2
II. La documentation du test d'intrusion avec CherryTree.....	2
Ex1 : Démarrage de Kali et recherche de CherryTree.....	3
Ex1 (suite) : Création de la section "Reconnaissance".....	3
Ex2 : Création de la section "Exploitation".....	4
Ex3 : Création de l'arborescence Reconnaissance / Recon active / Recon passive.....	5
Ex4 : Finalisation de l'arborescence.....	5
III. Effectuer une reconnaissance efficace avec SpiderFoot.....	6
1) Les outils Kali de reconnaissance sur une cible.....	6
Ex5 : Rappel du TP précédent et test de ping.....	7
2) Utilisation de SpiderFoot.....	8
a) Démarrage de SpiderFoot et affichage de l'aide.....	8
b) Élévation des privilèges.....	8
d) Lancement du serveur web SpiderFoot.....	9
e) Lancement d'un scan depuis le navigateur.....	10
f) Suivi du scan en cours d'exécution.....	11
g) Comparaison avec un scan complet ("All").....	12
h) Exploration des options du scan.....	13
Ex6 : Recherche d'un module tiers (HackerOne).....	14
i) Résultats détaillés du scan.....	14
Ex7 : Consignes de sécurité.....	15
Travail à faire (à réaliser).....	15
Conclusion.....	16

Introduction

Dans ce TP, j'ai travaillé sur la reconnaissance lors d'un test d'intrusion avec Kali Linux. Le but était de comprendre comment récupérer des informations sur une machine avant de chercher d'éventuelles failles.

J'ai utilisé CherryTree pour organiser mes recherches et mes résultats. Enfin, j'ai découvert SpiderFoot, qui permet d'automatiser la recherche d'informations sur une cible. Toutes les manipulations ont été réalisées sur mes machines virtuelles afin de rester dans un environnement de test.

Préparation

Avant de commencer ce TP, j'ai créé une machine virtuelle Windows Seven SP1 Ultimate qui va servir de machine "victime" pour les tests d'intrusion, en complément de la machine Kali Linux ("attaquant") déjà utilisée lors du TP précédent !

J'ai vérifié que les deux machines virtuelles sont bien placées dans le même réseau afin de pouvoir communiquer entre elles pour la suite des travaux pratiques : la VM Kali Linux (attaquant) a pour adresse **IP 172.30.12.50**, et la VM Windows Seven SP1 (victime) a pour adresse **IP 172.30.12.51**.

I. Les deux types de reconnaissance

Dans cette première partie, j'ai étudié la phase de reconnaissance d'un test d'intrusion, qui consiste à récolter le maximum d'informations sur la cible. J'ai retenu qu'on distingue deux approches complémentaires :

a) La reconnaissance passive

J'ai compris que la reconnaissance passive est une collecte d'informations qui ne fait aucune interaction directe avec la cible (pas de scan, pas d'envoi de paquets depuis ma machine). Elle consiste à rechercher des informations publiquement accessibles :

- sur les moteurs de recherche et les réseaux sociaux (noms des employés, emails, mots de passe compromis via des fuites de données)
- sur des forums techniques, pour identifier les technologies utilisées par l'entreprise (version de PHP, de Django, du SGBD, etc.)

Avantage : le potentiel de recherche est quasiment illimité. Inconvénient : les résultats obtenus restent limités en profondeur.

b) La reconnaissance active

J'ai vu que la reconnaissance active implique cette fois une interaction directe avec la cible, via des outils comme nmap ou nessus. Elle permet d'obtenir des informations plus précises : protocoles utilisés, services actifs, ports ouverts/fermés/filtrés.

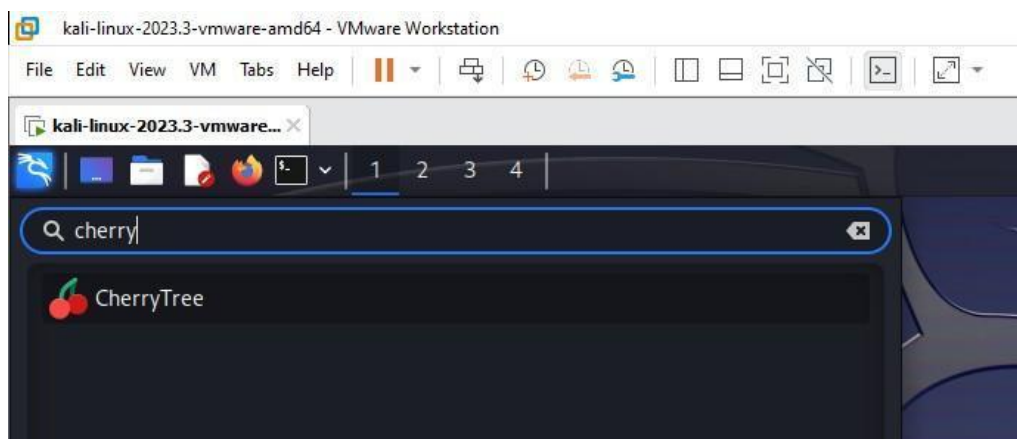
Avantage : on obtient beaucoup plus d'informations. Inconvénient : on peut être détecté et bloqué par un pare-feu ou par un système de protection si l'on envoie trop de paquets suspects.

II. La documentation du test d'intrusion avec CherryTree

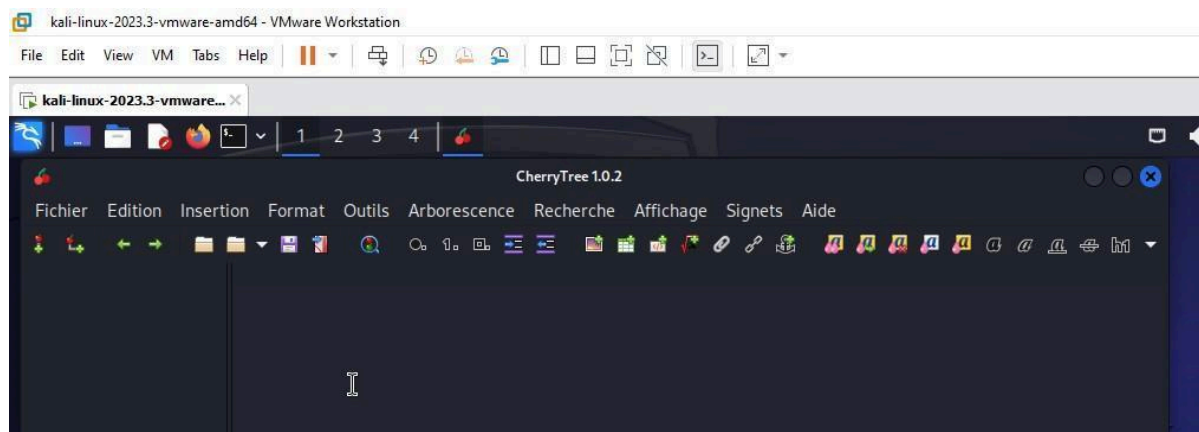
J'ai appris qu'il est indispensable de documenter chaque étape d'un test d'intrusion (commandes exécutées, outils utilisés, résultats obtenus) afin de faciliter la rédaction du rapport final. J'ai utilisé pour cela l'outil CherryTree, préinstallé sur Kali Linux.

Ex1 : Démarrage de Kali et recherche de CherryTree

J'ai démarré ma machine Kali Linux, puis j'ai recherché et lancé l'application CherryTree.

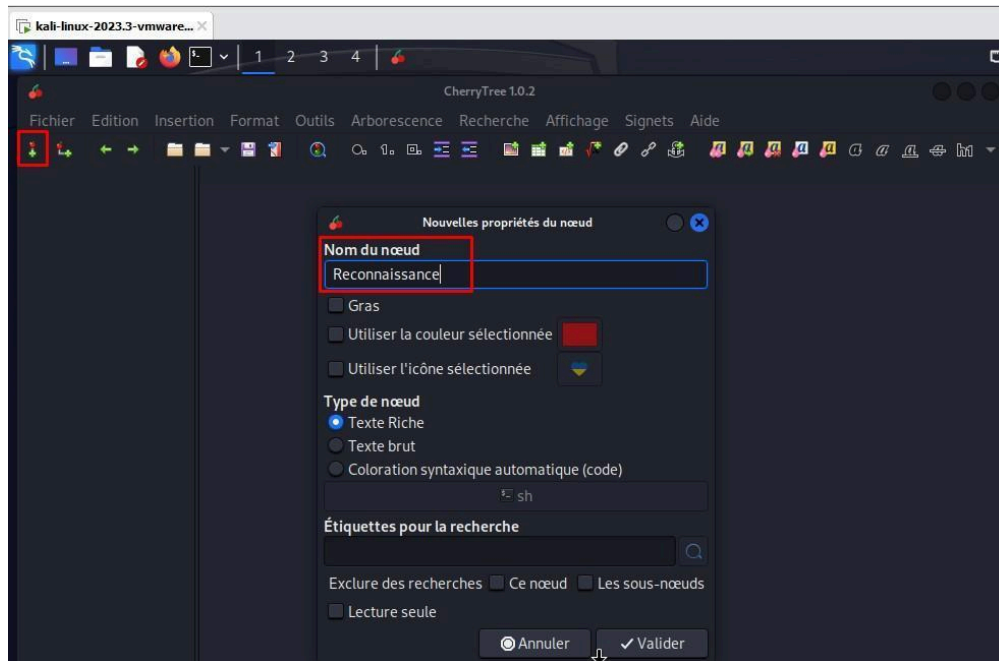


Recherche et lancement de CherryTree depuis le menu Kali



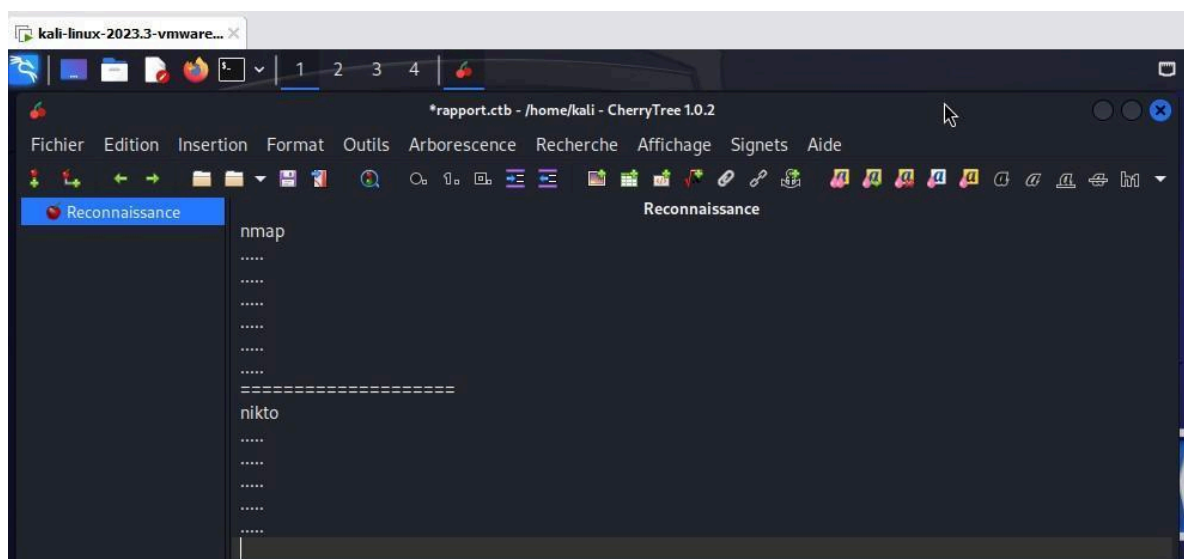
Ex1 (suite) : Création de la section "Reconnaissance"

J'ai créé un nouveau nœud nommé "Reconnaissance" en cliquant sur l'icône de création de nœud, puis j'ai renseigné le nom dans la boîte de dialogue "Nouvelles propriétés du nœud".



Création du nœud "Reconnaissance"

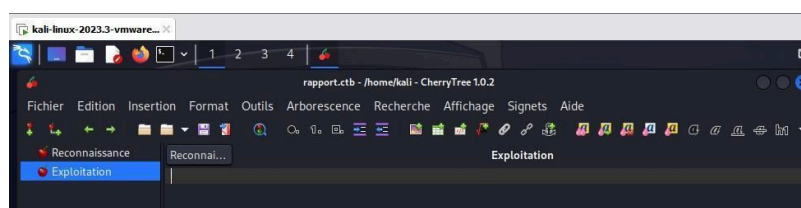
J'ai ensuite pu écrire dans ce nœud les résultats des différents scans, par exemple ceux de nmap et de nikto, séparés par une ligne de séparation.



Contenu du nœud Reconnaissance : résultats de nmap et nikto

Ex2 : Création de la section "Exploitation"

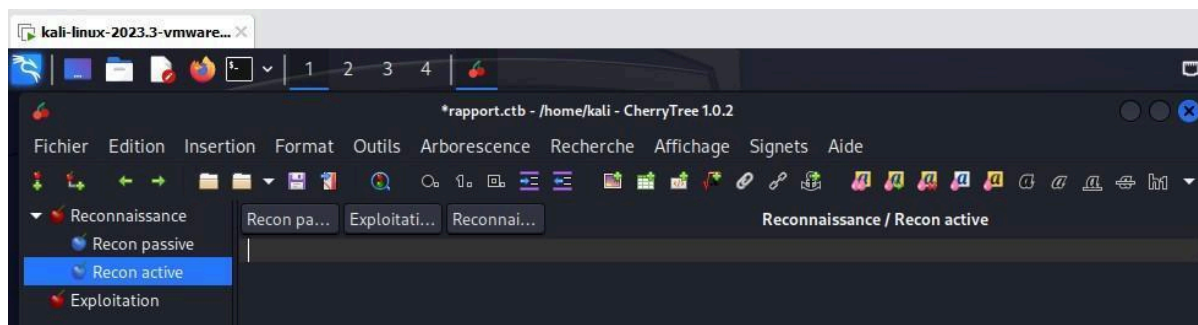
J'ai créé un second nœud nommé "Exploitation", au même niveau que le nœud "Reconnaissance", afin de séparer les deux grandes phases du test d'intrusion.



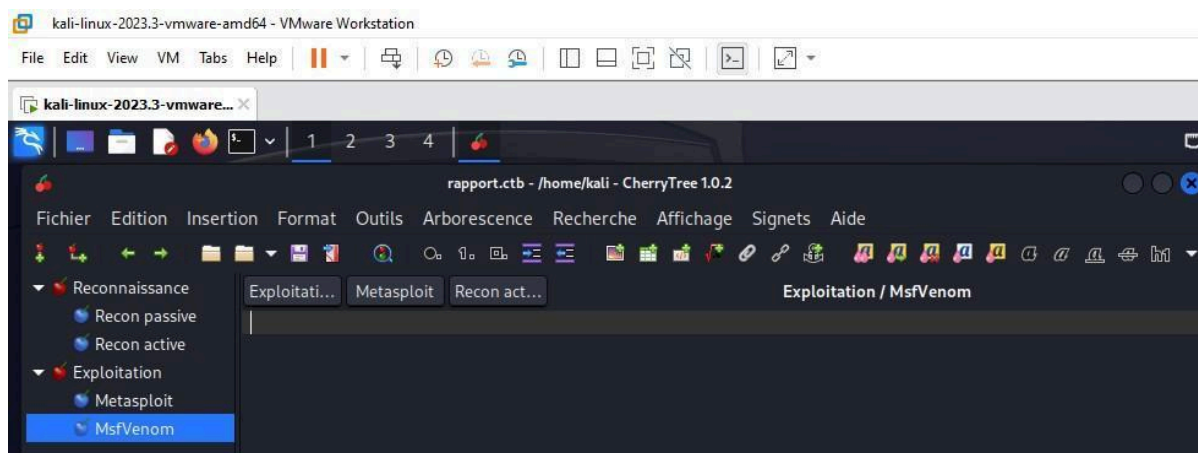
Ajout du nœud "Exploitation"

Ex3 : Création de l'arborescence Reconnaissance / Recon active / Recon passive

J'ai construit l'arborescence demandée en ajoutant des sous-nœuds "Recon active" et "Recon passive" sous le nœud "Reconnaissance", puis j'ai déplacé "Recon passive" pour obtenir l'ordre correct.



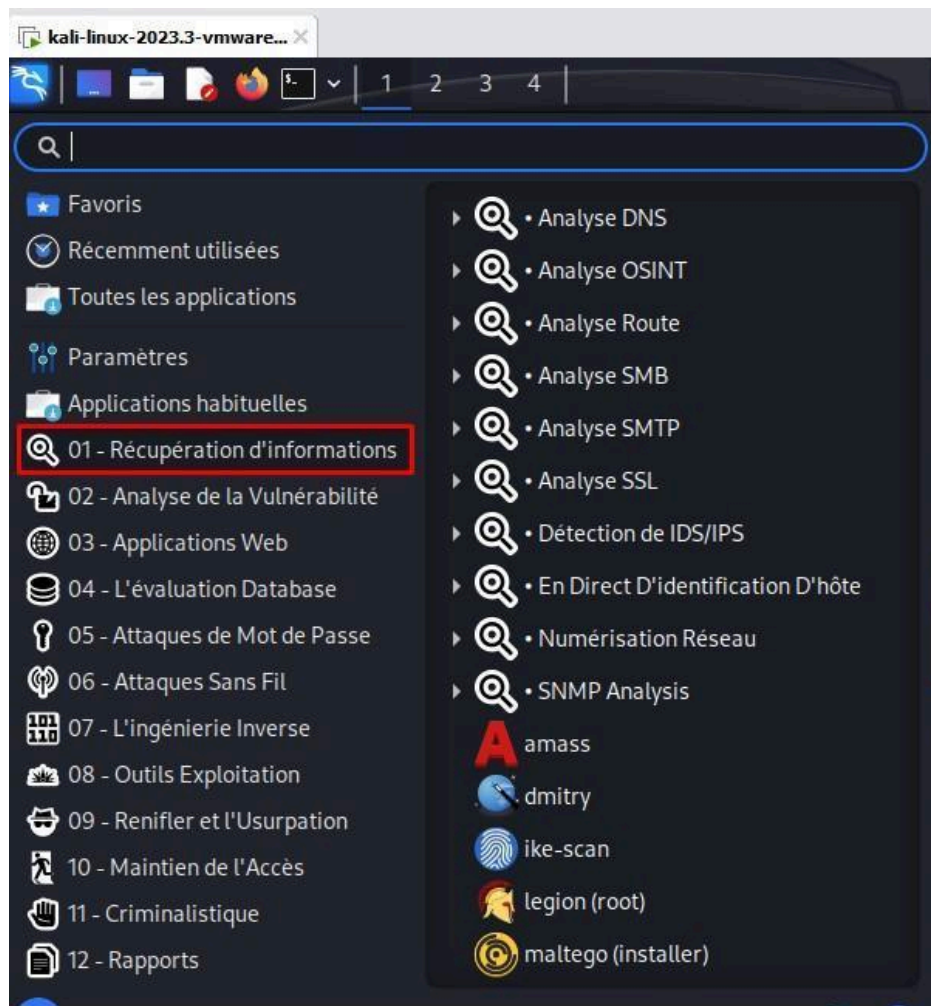
Arborescence intermédiaire avant déplacement du nœud Recon passive



Arborescence finale : Reconnaissance > Recon passive / Recon active

Ex4 : Finalisation de l'arborescence

J'ai complété l'arborescence en ajoutant sous "Exploitation" les sous-nœuds "Metasploit" et "MsfVenom", pour obtenir une organisation complète et prête à recevoir les résultats de mes prochains tests.



Arborescence finalisée du rapport CherryTree

Grâce à CherryTree, j'ai maintenant un fichier de rapport (rapport.ctb) organisé, qui va me permettre de retrouver facilement toutes les informations récoltées et de faciliter la rédaction du rapport final du test d'intrusion.

III. Effectuer une reconnaissance efficace avec SpiderFoot

1) Les outils Kali de reconnaissance sur une cible

Je suis allé sur Kali dans la zone "01 - Récupération d'informations", qui regroupe une quantité d'outils permettant de faire du scanning passif et actif sur un réseau ou un système cible.



Zone "01 - Récupération d'informations" et sous-catégories (Analyse DNS, etc.)

J'ai parcouru les différentes sections et voici à quoi servent les outils :

- **Analyse DNS** ([dnsenum](#), [dnsrecon](#), [fierce](#)) : permettent d'obtenir des informations sur les noms de domaine et les adresses IP.
- **Détection IDS/IPS** ([lbd](#), [wafw00f](#)) : permettent de vérifier si une cible est protégée par un pare-feu ou un système de sécurité.
- **Identification d'hôtes** ([arping](#), [fping](#), [hping3](#), [masscan](#), [netcat](#), [thcping6](#)) : permettent de savoir si une machine est active sur le réseau.
- **Scan réseau** ([masscan](#), [nmap](#)) : permettent d'analyser les machines présentes sur un réseau et d'obtenir des informations sur leurs systèmes.
- **Analyse OSINT** ([maltego](#), [spiderfoot](#), [spiderfoot-cli](#), [theharvester](#)) : permettent de récupérer automatiquement des informations publiques sur une cible.
- **Analyse des routes** ([netdiscover](#), [netmask](#)) : permettent d'obtenir des informations sur la structure et les routes d'un réseau.
- **Analyse SMB** ([enum4linux](#), [nbtscan](#), [smbmap](#)) : permettent d'analyser le protocole SMB, principalement utilisé dans les environnements Microsoft.
- **Analyse SMTP** ([swaks](#)) : permet de tester et d'analyser le protocole utilisé pour l'envoi des e-mails.
- **Analyse SNMP** ([onesixtyone](#), [snmp-check](#)) : permettent de détecter le protocole SNMP et d'obtenir des informations sur sa version.
- **Analyse SSL** ([ssldump](#), [ssllh](#), [ssllscan](#), [sslyze](#)) : permettent de vérifier les versions et la configuration SSL utilisées par un serveur, notamment pour identifier d'éventuels problèmes de sécurité.

Ex5 : Rappel du TP précédent et test de ping

Je me suis rappelé du TP précédent (B3 TP2.2), dans lequel j'avais préparé les deux machines nécessaires : l'attaquant (Kali Linux) et la victime (Windows Seven SP1 Ultimate).

B3 TP2.2 : Test d'intrusion : les machines nécessaires

On a vu la méthodologie que l'on va utiliser pour effectuer nos tests d'intrusion.
Pour faire cela, on a besoin de quelques machines virtuelles :

- L'attaquant : kali Linux,
- La victime : Windows Seven SP1 Ultimate

Ex1 : préparer ces machines et vérifier qu'elles sont dans le même réseau

a) Depuis Kali, en mode console, j'ai pingué la machine Windows (172.30.12.51) avec la commande ping 172.30.12.51. Le ping n'a pas répondu.

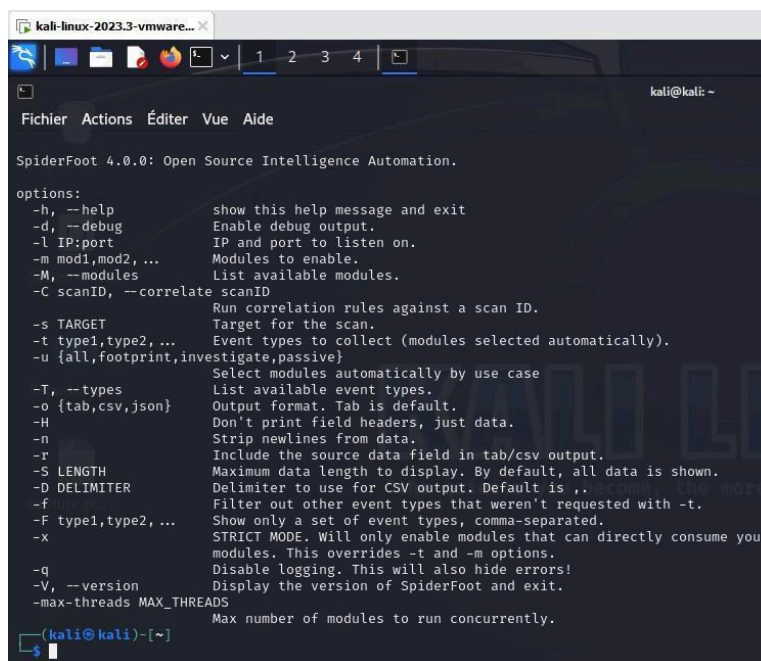
En effet, les machines Windows sont conçues pour ne pas répondre au ping, car celui-ci est un des premiers éléments utilisés par les hackers éthiques et testeurs d'intrusion pour détecter les services, ports et protocoles utilisés. C'est donc un moyen de protection mis en place par Microsoft. J'ai bien noté que le fait qu'une machine ne réponde pas au ping ne signifie pas qu'elle n'est pas active.

2) Utilisation de SpiderFoot

J'ai ensuite mis en pratique un outil de reconnaissance automatisée : SpiderFoot, préinstallé sur Kali Linux.

a) Démarrage de SpiderFoot et affichage de l'aide

J'ai ouvert un terminal sur Kali et démarré SpiderFoot afin d'afficher son aide et ses options.



```
kali-linux-2023.3-vmware...
Fichier Actions Éditer Vue Aide
kali@kali: ~
SpiderFoot 4.0.0: Open Source Intelligence Automation.

options:
-h, --help            show this help message and exit
-d, --debug           Enable debug output.
-l IP:port            IP and port to listen on.
-m mod1,mod2, ...    Modules to enable.
-M, --modules         List available modules.
-C scanID, --correlate scanID
                     Run correlation rules against a scan ID.
-s TARGET             Target for the scan.
-t type1,type2, ...   Event types to collect (modules selected automatically).
-u {all,footprint,investigate,passive}
                     Select modules automatically by use case
-T, --types           List available event types.
-o {tab,csv,json}     Output format. Tab is default.
-H                   Don't print field headers, just data.
-n                   Strip newlines from data.
-r                   Include the source data field in tab/csv output.
-S LENGTH            Maximum data length to display. By default, all data is shown.
-D DELIMITER         Delimiter to use for CSV output. Default is ,.
-f                   Filter out other event types that weren't requested with -t.
-F type1,type2, ...   Show only a set of event types, comma-separated.
-x                   STRICT MODE. Will only enable modules that can directly consume your
                     modules. This overrides -t and -m options.
-q                   Disable logging. This will also hide errors!
-V, --version         Display the version of SpiderFoot and exit.
-max-threads MAX_THREADS
                     Max number of modules to run concurrently.
```

Aide de SpiderFoot 4.0.0 (options disponibles)

b) Élévation des privilèges

J'ai élevé mes privilèges avec la commande sudo su afin de pouvoir lancer le serveur web de SpiderFoot.

```
(kali@kali)-[~]
$ sudo su
[sudo] Mot de passe de kali :
(root@kali)-[/home/kali]
```

Élévation des privilèges (sudo su)

c) J'ai identifié que l'option `-l` permet de préciser l'adresse IP et le port sur lesquels le serveur web de SpiderFoot va écouter.

d) Lancement du serveur web SpiderFoot

J'ai tapé la commande suivante pour ignorer les erreurs de vérification SSL :

```
export PYTHONWARNINGS="ignore:Unverified HTTPS request"
```

```
(root@kali)-[/home/prof]
# export PYTHONWARNINGS="ignore:Unverified HTTPS request"
```

Puis j'ai récupéré l'adresse IP de ma machine Kali (172.30.12.50) et j'ai lancé le serveur web de SpiderFoot sur le port 80 avec la commande `spiderfoot -l` suivie de l'adresse IP et du port :

```
spiderfoot -l 172.30.12.50:80
```

```
(kali@kali)-[~]
$ sudo su
[sudo] Mot de passe de kali :
(root@kali)-[/home/kali]
# spiderfoot -l 172.30.12.50:80
```

Lancement du serveur SpiderFoot avec `sudo su` et `spiderfoot -l`

J'ai constaté que le serveur web s'est bien lancé, avec un message m'indiquant l'URL à utiliser dans mon navigateur (<http://172.30.12.50:80/>). Le terminal continue de fonctionner en arrière-plan.

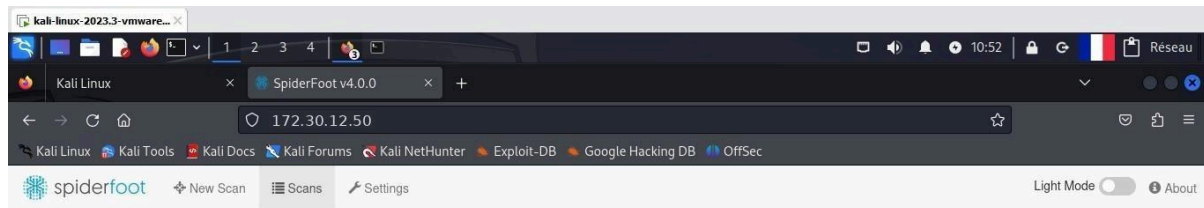
```
(kali@kali)-[~]
$ sudo su
[sudo] Mot de passe de kali :
(root@kali)-[/home/kali]
# spiderfoot -l 172.30.12.50:80
2024-03-08 10:48:43,085 [INFO] sf : Starting web server 172.30.12.50:80 ...

*****
2024-03-08 10:48:43,128 [WARNING] sf :
***** become, the
Warning: passwd file contains no passwords. Authentication disabled.
Please consider adding authentication to protect this instance!
Refer to https://www.spiderfoot.net/documentation/#security.
*****

Use SpiderFoot by starting your web browser of choice and
browse to http://172.30.12.50:80/
*****
```

e) Lancement d'un scan depuis le navigateur

J'ai ouvert mon navigateur à l'adresse indiquée. Au premier accès, la page "Scans" ne contenait aucun historique.



Scans

No scan history

There is currently no history of previously run scans. Please click 'New Scan' to initiate a new scan.

Page Scans de SpiderFoot, sans historique

J'ai cliqué sur "New Scan" pour créer un nouveau scan. J'ai renseigné un nom de scan (Scan Name) et la cible (Scan Target), qui peut être un nom de domaine, une adresse IP ou l'adresse d'un réseau.

Formulaire de création d'un nouveau scan

Sachant qu'un scan complet peut être très long, j'ai allégé la recherche en utilisant le mode "Investigate", adapté à une cible locale sur laquelle on veut obtenir des informations rapidement.

By Use Case By Required Data By Module

☒ All **Get anything and everything about the target.**
All SpiderFoot modules will be enabled (slow) but every possible piece of information about the target will be obtained and analysed.

☐ Footprint **Understand what information this target exposes to the Internet.**
Gain an understanding about the target's network perimeter, associated identities and other information that is obtained through a lot of web crawling and search engine use.

☐ Investigate **Best for when you suspect the target to be malicious but need more information.**
Some basic footprinting will be performed in addition to querying of blacklists and other sources that may have information about your target's maliciousness.

☐ Passive **When you don't want the target to even suspect they are being investigated.**
As much information will be gathered without touching the target or their affiliates, therefore only modules that do not touch the target will be enabled.

Sélection du mode Investigate

J'ai ensuite sélectionné uniquement quelques modules pertinents (onglet "By Module") plutôt que de tout activer, afin de réduire la durée du scan :

By Use Case By Required Data By Module Select All De-Select All

☒	AbstractAPI	Look up domain, phone and IP address information from AbstractAPI.
☒	abuse.ch	Check if a host/domain, IP address or netblock is malicious according to Abuse.ch.
☒	AbuseIPDB	Check if an IP address is malicious according to AbuseIPDB.com blacklist.
☒	Abusix Mail Intelligence	Check if a netblock or IP address is in the Abusix Mail Intelligence blacklist.
☒	Account Finder	Look for possible associated accounts on nearly 200 websites like Ebay, Slashdot, reddit, etc.
☒	AdBlock Check	Check if linked pages would be blocked by AdBlock Plus.
☒	AdGuard DNS	Check if a host would be blocked by AdGuard DNS.
☒	Ahmia	Search Tor 'Ahmia' search engine for mentions of the target.
☒	AlienVault IP Reputation	Check if an IP or netblock is malicious according to the AlienVault IP Reputation database.
☒	AlienVault OTX	Obtain information from AlienVault Open Threat Exchange (OTX)
☒	Amazon S3 Bucket Finder	Search for potential Amazon S3 buckets associated with the target and attempt to list their contents.
☒	Apple iTunes	Search Apple iTunes for mobile apps.
☒	Archive.org	Identifies historic versions of interesting files/pages from the Wayback Machine.
☒	ARIN	Queries ARIN registry for contact information.
☒	Azure Blob Finder	Search for potential Azure blobs associated with the target and attempt to list their contents.
☒	Bad Packets	Obtain information about any malicious activities involving IP addresses found
☒	Base64 Decoder	Identify Base64-encoded strings in URLs, often revealing interesting hidden information.
☒	BGPView	Obtain network information from BGPView API.
☒	Binary String Extractor	Attempt to identify strings in binary content.

♥ Join the SpiderFoot community Discord!

Liste des modules avec les boutons Select All / De-Select All

☒	Port Scanner - TCP	Scans for commonly open TCP ports on Internet-facing systems.
☒	Web Server Identifier	Obtain web server banners to identify versions of web servers being used.
☒	DNS Resolver	Resolves hosts and IP addresses identified, also extracted from raw content.
☒	Tool - nbtscan	Scans for open NETBIOS nameservers on your target's network.
☒	Tool - Nmap	Identify what Operating System might be used.
☒	DNS Raw Records	Retrieves raw DNS records such as MX, TXT and others.

Modules sélectionnés : Port Scanner TCP, Web Server Identifier, DNS Resolver, Tool nbtscan, Tool Nmap, DNS Raw Records

f) Suivi du scan en cours d'exécution

J'ai lancé le scan. Pendant que le scan s'exécutait, j'ai pu observer dans le terminal que celui-ci continuait de fonctionner en arrière-plan et affichait le détail de ses requêtes.

```

root@kali: /home/kali

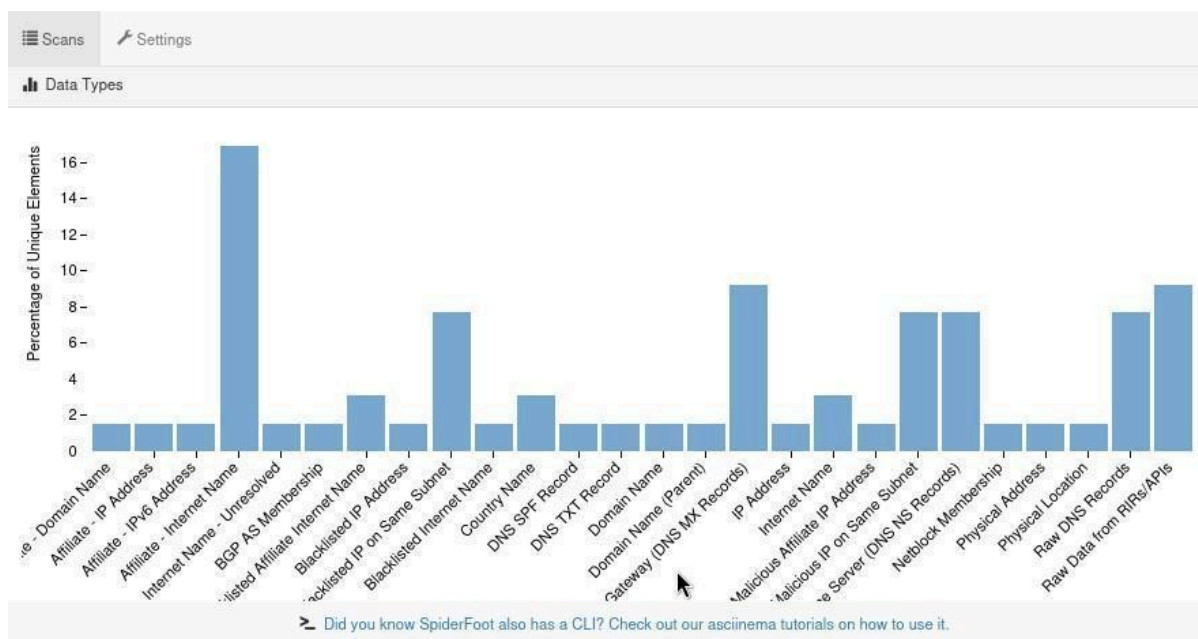
Fichier Actions Éditer Vue Aide

.0) Gecko/20100101 Firefox/62.0, timeout=5, cookies=None)
2024-03-08 11:28:42,781 [INFO] sflib : Fetching (GET): https://api.mnemonic.no/pdns/v3/192.168.0.4?limit=500&offset=0 (proxy=None, user-agent=Gecko/20100101 Firefox/62.0, timeout=30, cookies=None)
2024-03-08 11:28:43,017 [INFO] sflib : Fetched https://api.flickr.com/services/rest?sort=relevance&parse_tags=1&content_type=7&extras=description%2Crealname&hermes=1&hermesClient=1&reqId=6nojsoncallback=1&viewer=NSID=6method=flickr.photos.search&csrf=6lang=en-US&per_page=100&page=XXX (128 bytes in 0.7848341464996338s)
2024-03-08 11:28:43,186 [INFO] sflib : Fetched https://threatfox-api.abuse.ch/api/v1/ (88 bytes in 0.6060140132904053s)
2024-03-08 11:28:44,023 [INFO] sfp_flickr : Parsing page 1 of 0
2024-03-08 11:28:44,023 [ERROR] sfp_fullcontact : You enabled sfp_fullcontact but did not set an API key!
2024-03-08 11:28:44,024 [INFO] sflib : Fetching (GET): https://sonar.omnisint.io/subdomains/btsssiogap.com?page=0 (proxy=None, user-agent=Gecko/20100101 Firefox/62.0, timeout=30, cookies=None)
2024-03-08 11:28:44,191 [INFO] sflib : Fetching (GET): https://en.wikipedia.org/w/api.php?action=feedcontributions&user=XXX (proxy=None, user-agent=Gecko/20100101 Firefox/62.0, timeout=5, cookies=None)
2024-03-08 11:28:45,108 [INFO] sflib : Fetched https://en.wikipedia.org/w/api.php?action=feedcontributions&user=XXX (431 bytes in 0.9171011284444444s)
2024-03-08 11:28:46,059 [INFO] sflib : Fetched https://btsssiogapcomspace.nyc3.digitaloceanspaces.com (250 bytes in 5.733612060546875s)
2024-03-08 11:28:46,376 [INFO] sfp_digitaloceanspace : Spawning thread to check bucket: https://btsssiogapcomdata.nyc3.digitaloceanspaces.com
2024-03-08 11:28:46,380 [INFO] sfp_digitaloceanspace : Spawning thread to check bucket: https://btsssiogapcomprod.nyc3.digitaloceanspaces.com
2024-03-08 11:28:46,383 [INFO] sfp_digitaloceanspace : Spawning thread to check bucket: https://btsssiogapcomstaging.nyc3.digitaloceanspaces.com
2024-03-08 11:28:47,072 [INFO] sflib : Fetched https://btsssiogapcomprod.nyc3.digitaloceanspaces.com (249 bytes in 0.690941572189331s)
2024-03-08 11:28:47,091 [INFO] sflib : Fetched https://btsssiogapcomstaging.nyc3.digitaloceanspaces.com (252 bytes in 0.7070329189300537s)
2024-03-08 11:28:47,101 [INFO] sflib : Fetched https://btsssiogapcomdata.nyc3.digitaloceanspaces.com (249 bytes in 0.7226169109344482s)
2024-03-08 11:28:47,407 [INFO] sfp_digitaloceanspace : Spawning thread to check bucket: https://btsssiogapcomproduction.nyc3.digitaloceanspaces.com
2024-03-08 11:28:47,411 [INFO] sfp_digitaloceanspace : Spawning thread to check bucket: https://btsssiogapcomstage.nyc3.digitaloceanspaces.com
2024-03-08 11:28:47,415 [INFO] sfp_digitaloceanspace : Spawning thread to check bucket: https://btsssiogapcomapp.nyc3.digitaloceanspaces.com
2024-03-08 11:28:48,151 [INFO] sflib : Fetched https://btsssiogapcomproduction.nyc3.digitaloceanspaces.com (255 bytes in 0.7417664527893066s)
2024-03-08 11:28:48,222 [INFO] sflib : Fetched https://btsssiogapcomapp.nyc3.digitaloceanspaces.com (248 bytes in 0.8066542148590088s)
2024-03-08 11:28:48,379 [INFO] sflib : Fetched https://btsssiogapcomstage.nyc3.digitaloceanspaces.com (250 bytes in 0.9662487506866455s)
2024-03-08 11:28:48,668 [INFO] sflib : Fetching (GET): https://freeapi.robtext.com/ipquery/192.168.0.44 (proxy=None, user-agent=SpiderFoot, timeout=5, cookies=None)
2024-03-08 11:28:48,672 [INFO] sfp_digitaloceanspace : Spawning thread to check bucket: https://btsssiogapcommedia.nyc3.digitaloceanspaces.com
2024-03-08 11:28:48,673 [INFO] sfp_digitaloceanspace : Spawning thread to check bucket: https://btsssiogapcomdevelopment.nyc3.digitaloceanspaces.com
2024-03-08 11:28:48,677 [INFO] sfp_digitaloceanspace : Spawning thread to check bucket: https://btsssiogapcom-test.nyc3.digitaloceanspaces.com

```

Terminal affichant l'exécution du scan en arrière-plan

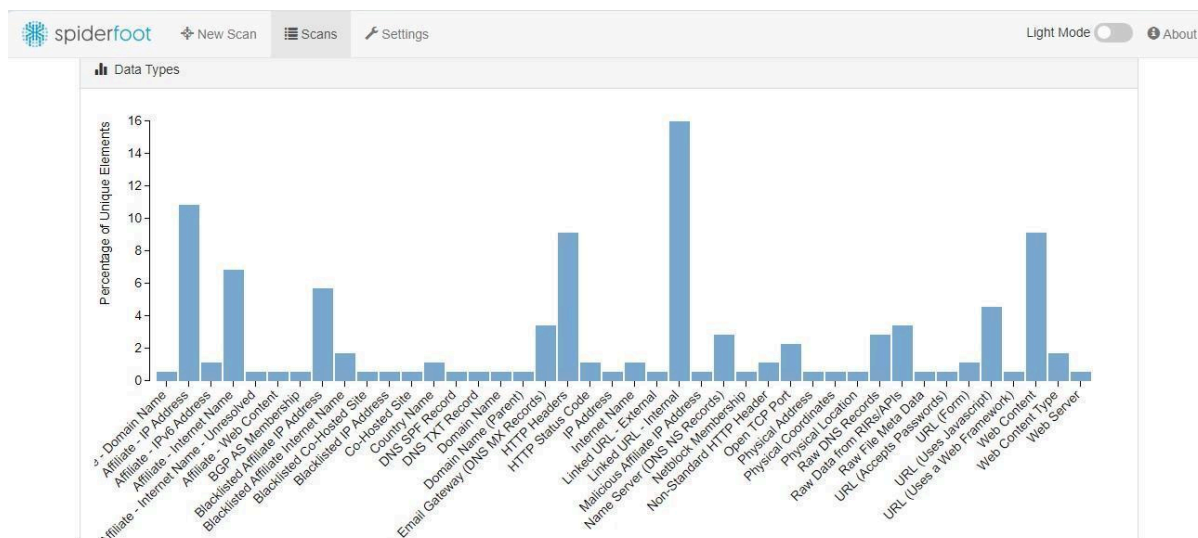
Une fois le scan terminé, j'ai affiché le graphique des résultats obtenus (répartition des types de données trouvées) :



Graphique des résultats du scan avec modules restreints

g) Comparaison avec un scan complet ("All")

J'ai ensuite relancé un scan en sélectionnant cette fois l'option "All", qui active tous les modules de SpiderFoot. J'ai constaté que j'ai obtenu beaucoup plus d'informations qu'avec la sélection restreinte de modules.



Graphique des résultats obtenus avec l'option All (tous les modules)

h) Exploration des options du scan

J'ai consulté l'onglet "By Required Data", qui permet de voir la liste de toutes les informations recherchées par SpiderFoot. J'ai noté que certaines informations ne sont récupérables qu'en mode actif, et d'autres uniquement en mode passif.

Line target or your scan:

Subnet: e.g. 1.2.3.0/24
Bitcoin Address: e.g. 1HesYJSP1QqcyPEjnQ9vzBL1wujrUNGe7R
Network ASN: e.g. 1234

By Use Case: **By Required Data** By Module

Select All De-Select All

☒	Account on External Site	☒	Affiliate - Company Name
☒	Affiliate - Domain Name	☒	Affiliate - Domain Name Unregistered
☒	Affiliate - Domain Whois	☒	Affiliate - Email Address
☒	Affiliate - IP Address	☒	Affiliate - IPv6 Address
☒	Affiliate - Internet Name	☒	Affiliate - Internet Name - Unresolved
☒	Affiliate - Internet Name Hijackable	☒	Affiliate - Web Content
☒	Affiliate Description - Abstract	☒	Affiliate Description - Category
☒	App Store Entry	☒	BGP AS Membership
☒	BGP AS Ownership	☒	Base64-encoded Data
☒	Bitcoin Address	☒	Bitcoin Balance
☒	Blacklisted Affiliate IP Address	☒	Blacklisted Affiliate Internet Name
☒	Blacklisted Co-Hosted Site	☒	Blacklisted IP Address

[Check out the SpiderFoot documentation to get more out of SpiderFoot.](#)

Onglet By Required Data

J'ai également consulté l'onglet "By Module", qui liste l'ensemble des modules (bases de données tierces) utilisés par SpiderFoot pour récolter le maximum d'informations.

☒	Gravatar	Retrieve user information from Gravatar API.
☒	Grayhat Warfare	Find bucket names matching the keyword extracted from a domain from Grayhat API.
☒	Greensnow	Check if a netblock or IP address is malicious according to greensnow.co.
☒	grep.app	Search grep.app API for links and emails related to the specified domain.
☒	GreyNoise	Obtain IP enrichment data from GreyNoise
☒	HackerOne (Unofficial)	Check external vulnerability scanning/reporting service h1.nobbd.de to see if the target is listed.
☒	HackerTarget	Search HackerTarget.com for hosts sharing the same IP.
☒	Hash Extractor	Identify MD5 and SHA hashes in web content, files and more.
☒	HaveIBeenPwned	Check HaveIBeenPwned.com for hacked e-mail addresses identified in breaches.
☒	Host.io	Obtain information about domain names from host.io.
☒	Hosting Provider Identifier	Find out if any IP addresses identified fall within known 3rd party hosting ranges, e.g. Amazon, Azure, etc.
☒	Human Name Extractor	Attempt to identify human names in fetched content.

Liste des modules SpiderFoot, dont HackerOne

Ex6 : Recherche d'un module tiers (HackerOne)

J'ai repéré le module "HackerOne (Unofficial)", qui permet de vérifier si la cible est référencée sur le service externe de scan/reporting de vulnérabilités h1.nobbd.de. HackerOne est en réalité une plateforme de "bug bounty" : elle met en relation des entreprises et des chercheurs en sécurité (hackers éthiques), qui peuvent signaler des failles de sécurité trouvées sur leurs services en échange d'une récompense financière.

i) Résultats détaillés du scan

Sur la page principale de SpiderFoot, j'ai pu suivre l'avancement de mon scan. Au début du scan, seuls 3 éléments avaient été collectés :

Scans

Filter: None

☐	Name	Target	Started	Finished	Status	Elements	Correlations	Action
☐	test scan	btsslogap.com	2024-03-09 09:45:36	Not yet	RUNNING	3	0 0 0 0	

Scans 1 - 1 / 1 (1)

Scan en cours : 3 éléments collectés au démarrage

Au bout d'une heure, j'ai constaté que 362 éléments avaient été récoltés au total :

Scans

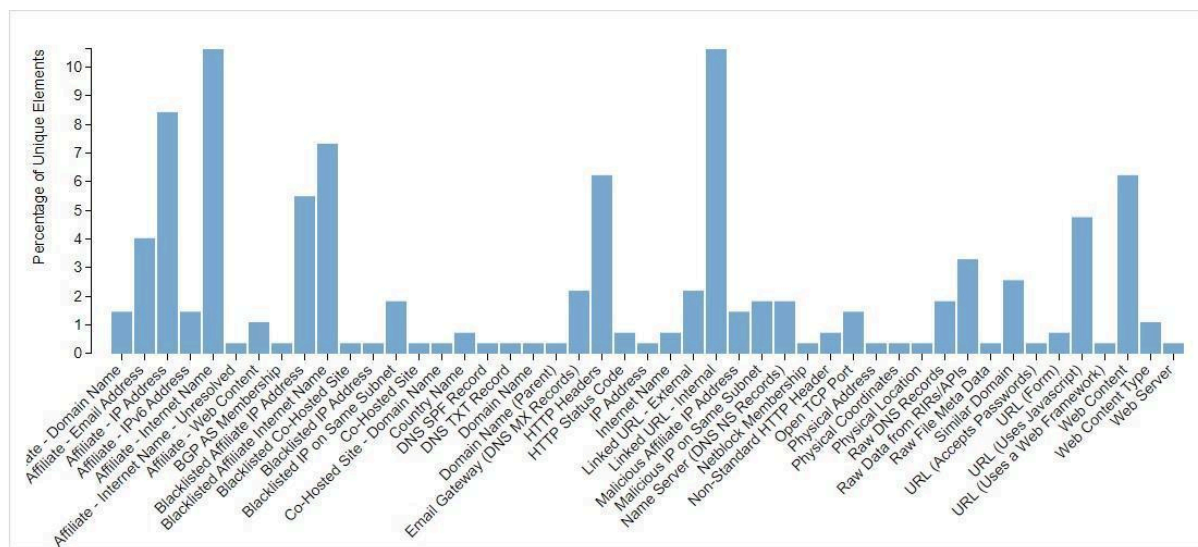
Filter: None

☐	Name	Target	Started	Finished	Status	Elements	Correlations	Action
☐	test scan	btsslogap.com	2024-03-09 09:45:36	Not yet	RUNNING	362	0 0 0 0	

Scans 1 - 1 / 1 (1)

Scan après une heure : 362 éléments collectés

J'ai ensuite affiché le graphique complet représentant la répartition de toutes ces données :



Graphique final de répartition des données collectées

Enfin, j'ai utilisé l'onglet "Browse" pour afficher la liste détaillée de tous les types de données récoltées (adresses IP, emails, noms de domaine affiliés, etc.), avec le nombre d'éléments uniques et le nombre total d'éléments pour chaque type :

Summary

Correlations

Browse

Graph

Scan Settings

Log

Refresh

Download

Search...

Type	Unique Data Elements	Total Data Elements	Last Data Element
Affiliate - Domain Name	4	14	2024-03-09 10:09:35
Affiliate - Email Address	11	11	2024-03-09 10:08:58
Affiliate - IP Address	23	26	2024-03-09 10:09:36
Affiliate - IPv6 Address	4	4	2024-03-09 10:09:36
Affiliate - Internet Name	29	41	2024-03-09 10:10:14
Affiliate - Internet Name - Unresolved	1	1	2024-03-09 09:46:48
Affiliate - Web Content	3	3	2024-03-09 10:07:00
BGP AS Membership	1	2	2024-03-09 09:55:56
Blacklisted Affiliate IP Address	15	15	2024-03-09 10:06:11
Blacklisted Affiliate Internet Name	20	20	2024-03-09 10:08:42
Blacklisted Co-Hosted Site	1	1	2024-03-09 10:01:13
Blacklisted IP Address	1	1	2024-03-09 09:54:55
Blacklisted IP on Same Subnet	5	5	2024-03-09 10:06:23

Onglet Browse : liste détaillée des données récoltées

J'ai bien noté que toutes ces informations sont précieuses et pourront être directement intégrées dans le rapport final du test d'intrusion, en les organisant par exemple dans CherryTree comme vu en partie II.

Ex7 : Consignes de sécurité

J'ai bien pris en compte l'avertissement du TP : un scan de reconnaissance active ne doit en aucun cas être effectué sur une cible qui ne m'appartient pas, car cela pourrait la faire planter ("crasher"). Je n'ai donc travaillé que sur mon propre environnement (serveur web local / réseau de mes machines virtuelles).

Travail à faire (à réaliser)

- Effectuer un scan SpiderFoot sur un serveur web local (ma propre VM).

- Noter les informations récoltées dans le fichier CherryTree, sous la section Reconnaissance.

Conclusion

Ce TP m'a permis de mieux comprendre la phase de reconnaissance d'un test d'intrusion. J'ai appris la différence entre la reconnaissance passive et active et j'ai découvert plusieurs outils de Kali Linux permettant de récupérer des informations.

J'ai également appris à utiliser CherryTree pour organiser mon travail et SpiderFoot pour automatiser les recherches. J'ai pu voir qu'un scan complet permet de récupérer beaucoup plus d'informations qu'un scan avec seulement quelques modules.

Ce TP m'a donc permis de mieux comprendre l'importance de la reconnaissance avant de passer aux étapes suivantes d'un test d'intrusion.